DISTRIBUTED SYSTEMS

Lab 8

Nuno Preguiça, Sérgio Duarte, Kevin Gallagher, Salvatore Signorello, João Bordalo, Dina Borrego

GOALS

In the end of this lab you should be able to:

- Understand what is HTTPS and SSL/TLS
- Know how to generate a keystore with server cryptographic keys
- Know how to generate a truststore with root certificates and the certificate for your server
- Know how to develop a REST server using https in Java
- Know how to develop a gRPC server using TLS in Java
- Know how to modify your REST clients to use https
- Know how to modify your gRPC clients to use TLS

GOALS

In the end of this lab you should be able to:

- **Understand what is HTTPS and SSL/TLS**
- Know how to generate a keystore with server cryptographic keys
- Know how to generate a truststore with root certificates and the certificate for your server
- Know how to develop a REST server using https in Java
- Know how to develop a gRPC server using TLS in Java
- Know how to modify your REST clients to use https
- Know how to modify your gRPC clients to use TLS

SSL/TLS

TLS is a cryptographic protocol suite to ensure secure communications over insecure networks. It is the second version of the SSL protocol (that was discovered to have vulnerabilities some years ago).

- TLS in addition to secure the communication also allows to authenticate the server.
- Optionally, it can also authenticate the client (this is a feature rarely used that we will not explore in this course)
- TLS is the basis for the HTTPS protocol that allows secure accesses to content and services in the web.

SSL/TLS

TLS is a cryptographic protocol suite to ensure secure communications over insecure networks. It is the second version of the SSL protocol (that was discovered to have vulnerabilities some years ago).

- T
  a
- C
  r
- T
  a

In summary, TLS provides the following security guarantees:

Confidentiality - data streams are encrypted;
Integrity - data streams are protected against tampering;
Authentication - identity of servers and clients (optionally) is asserted.

HTTPS

HTTPS is the name of the protocol that results from the exchange of HTTP messages on top of TLS (secure) connections.

It is now a standard for accessing content in the web:

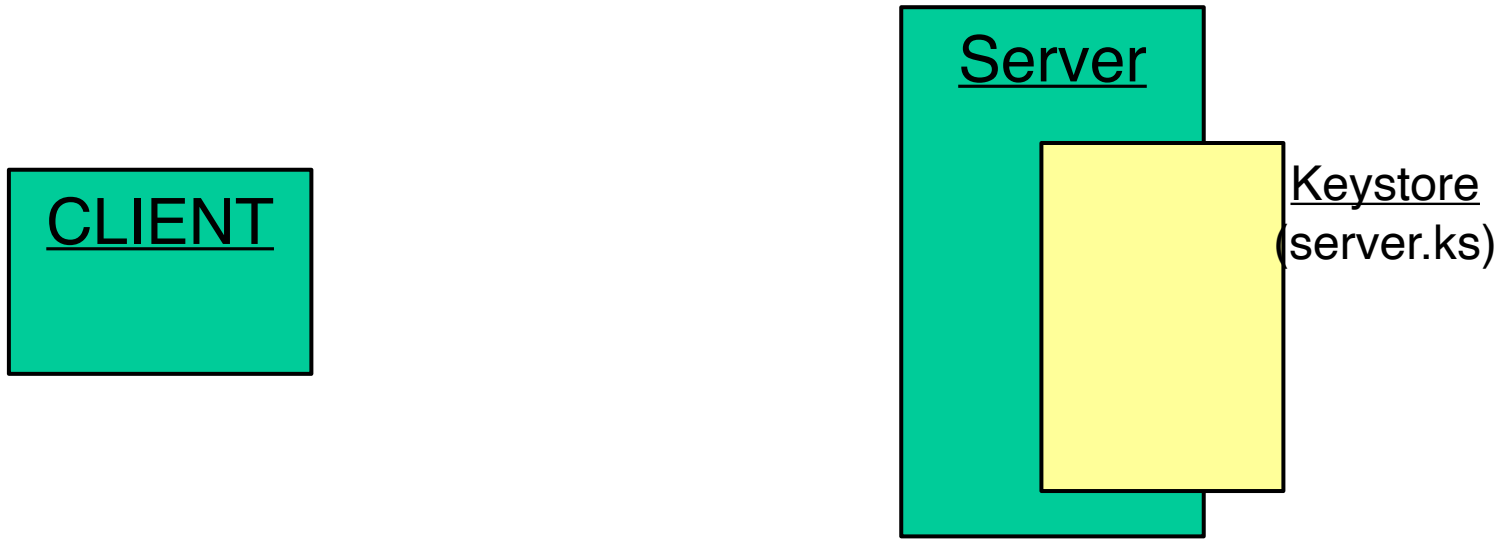
- It verifies the identity of the server, mitigating attacks such as Phishing and Man-in-the-Middle
- It ensures the confidentiality and integrity of all data exchanged between clients and servers (both requests and replies)

Important for instance, to enable sending passwords in URLs (as these will not be observable by someone inspecting the traffic departing the client machine)

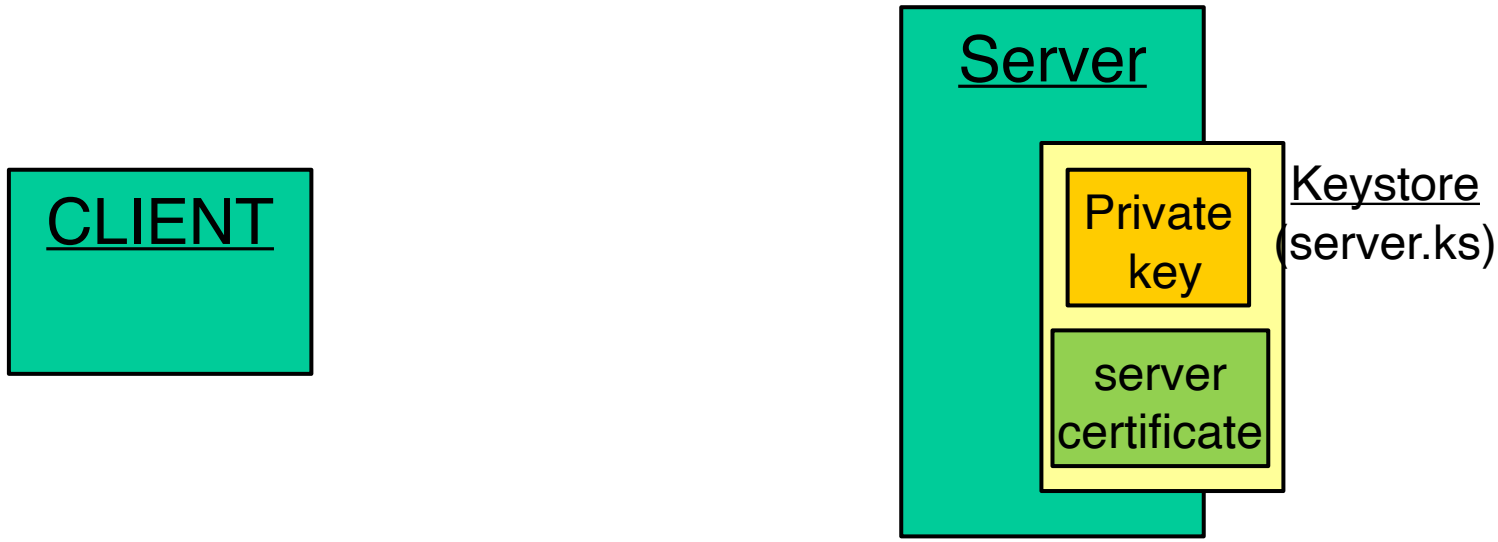
HTTPS (High Level Overview)



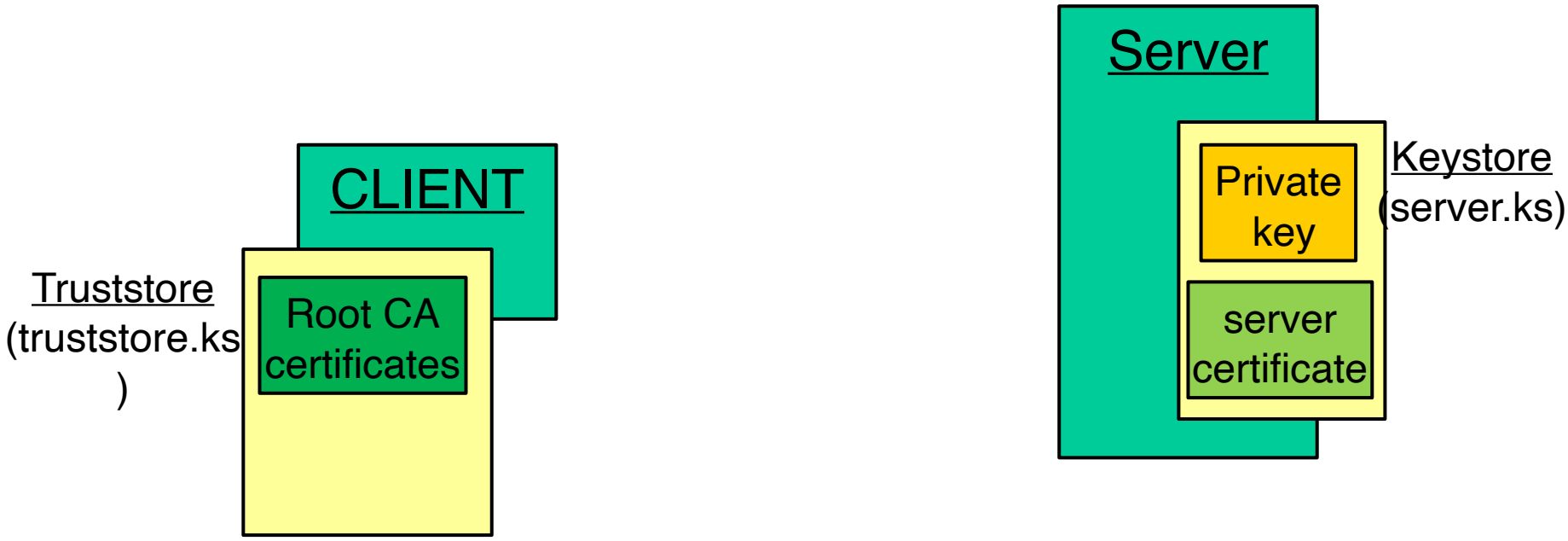
HTTPS (High Level Overview)



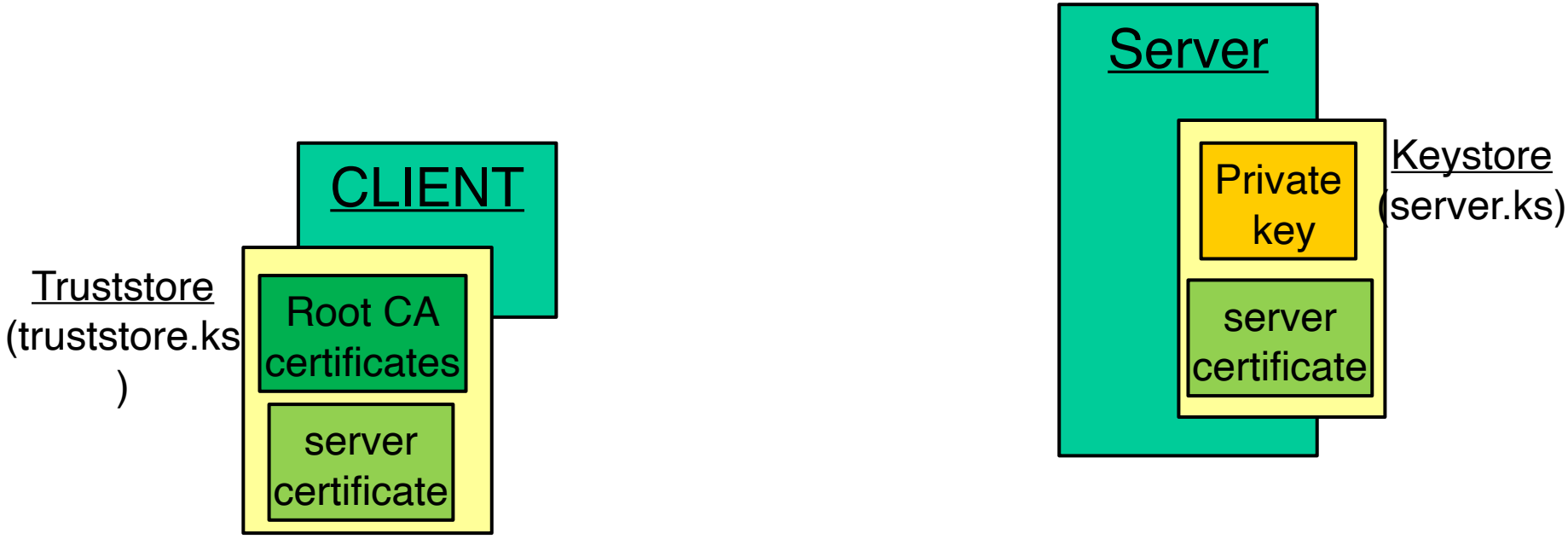
HTTPS (High Level Overview)



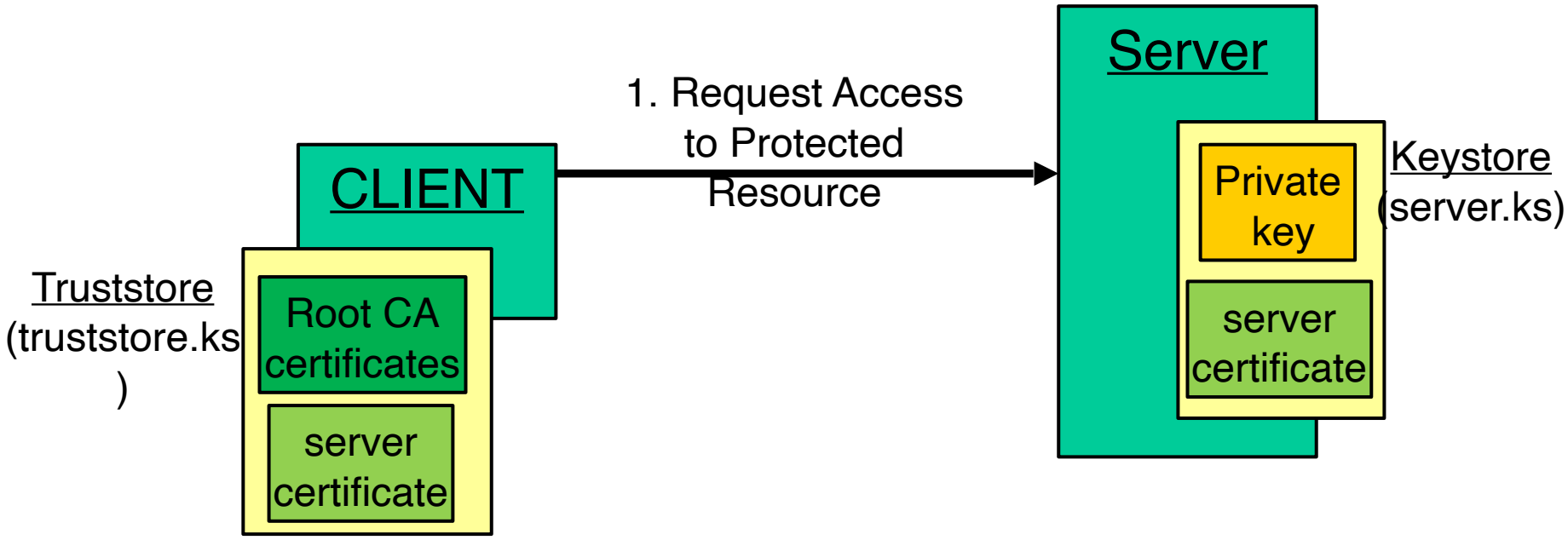
HTTPS (High Level Overview)



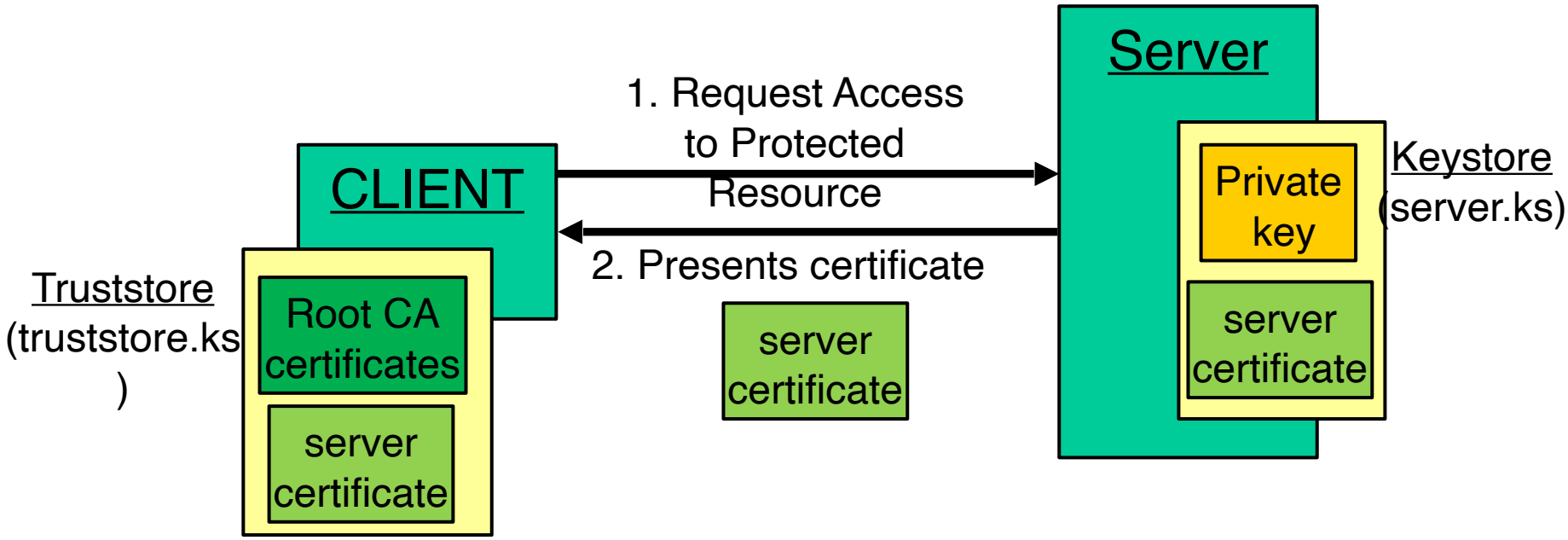
HTTPS (High Level Overview)



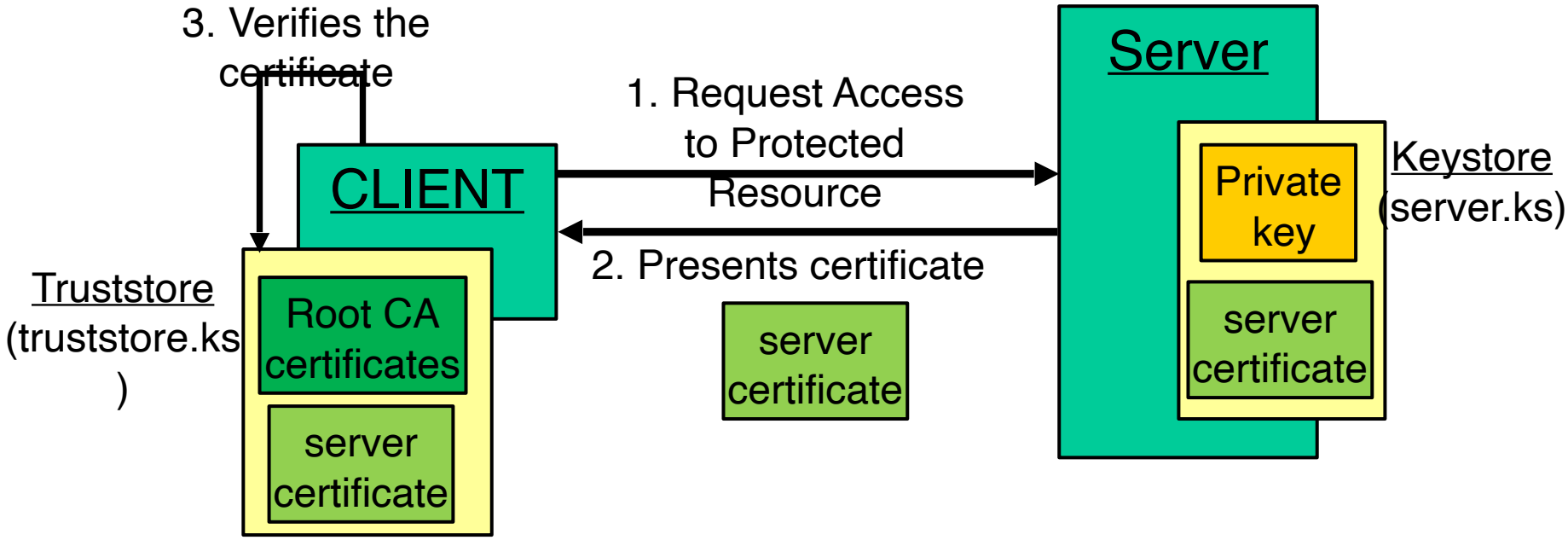
HTTPS (High Level Overview)



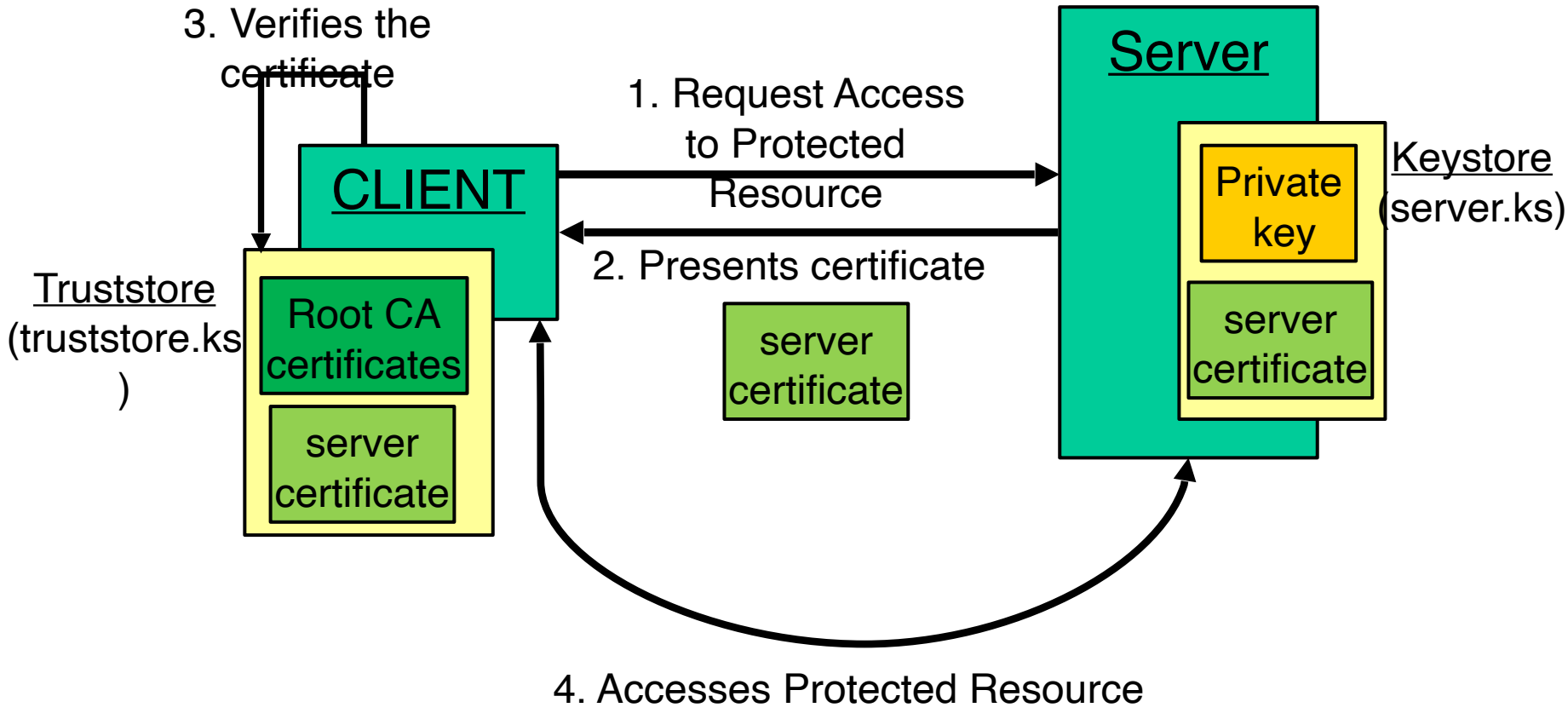
HTTPS (High Level Overview)



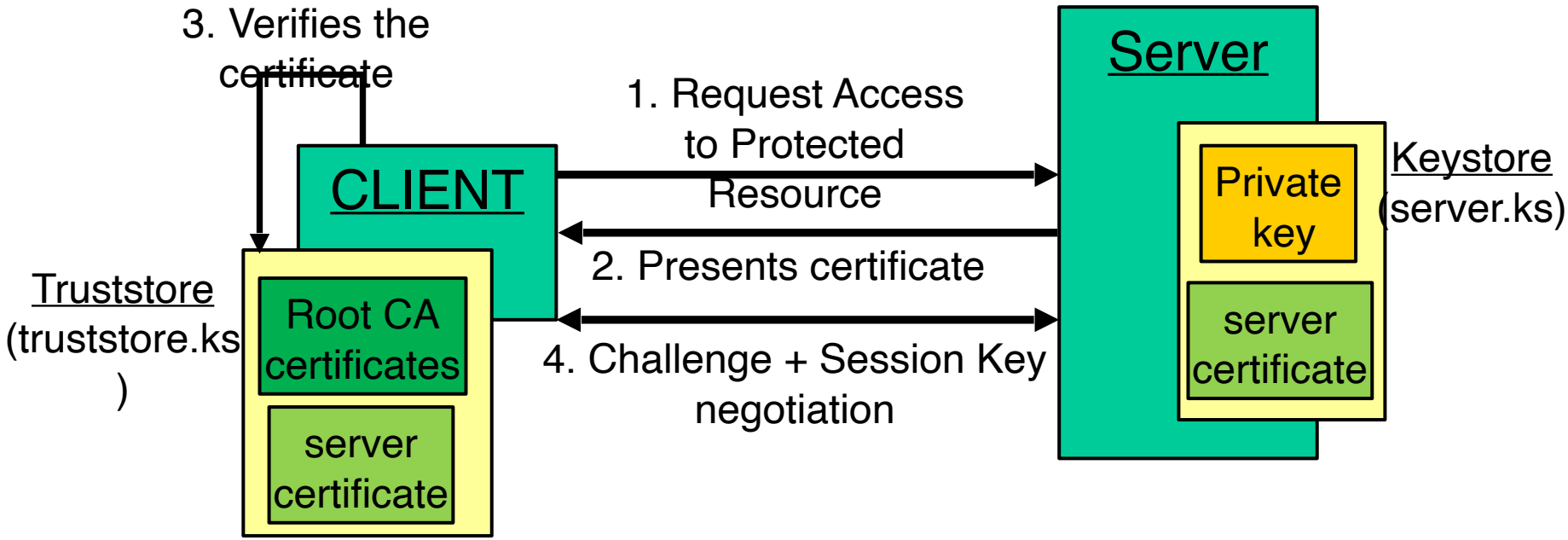
HTTPS (High Level Overview)



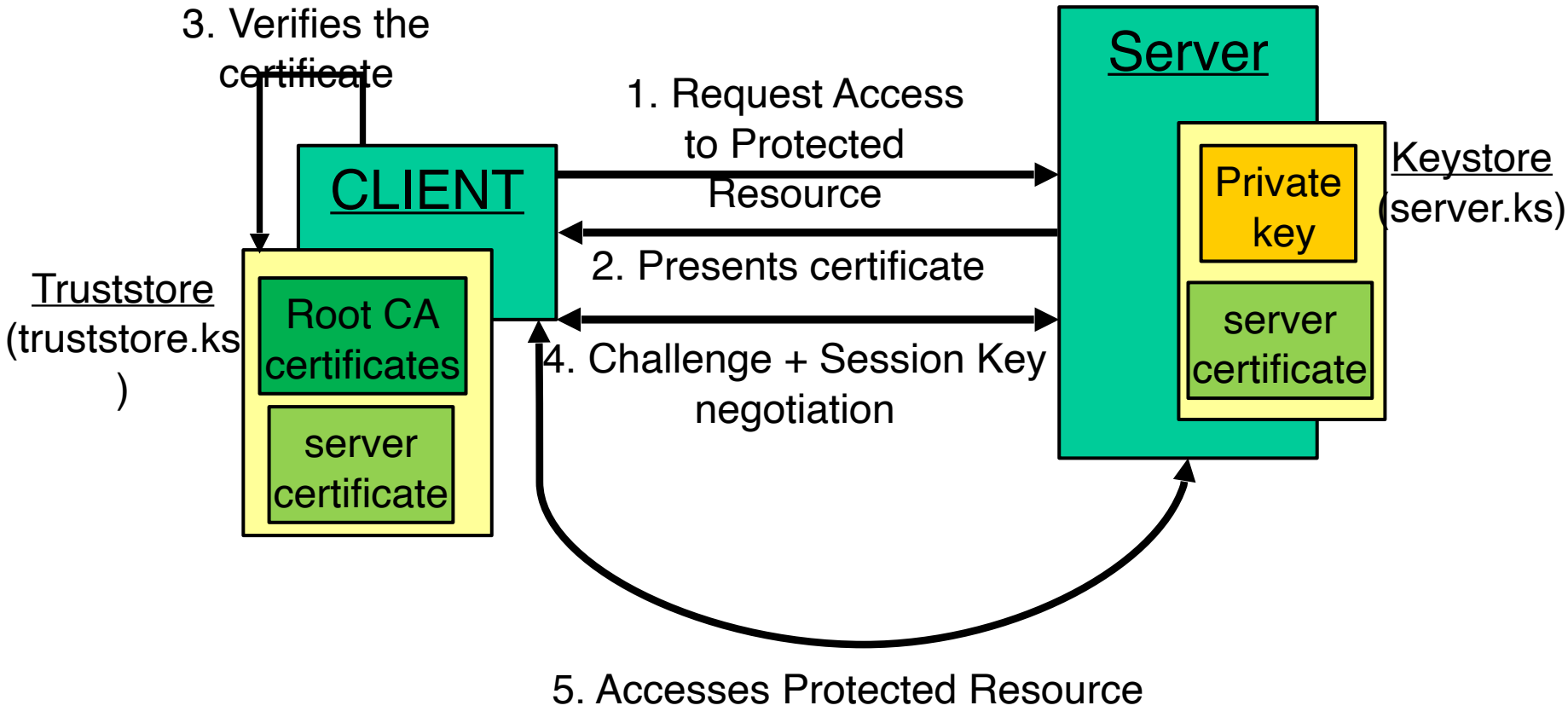
HTTPS (High Level Overview)



HTTPS (High Level Overview)

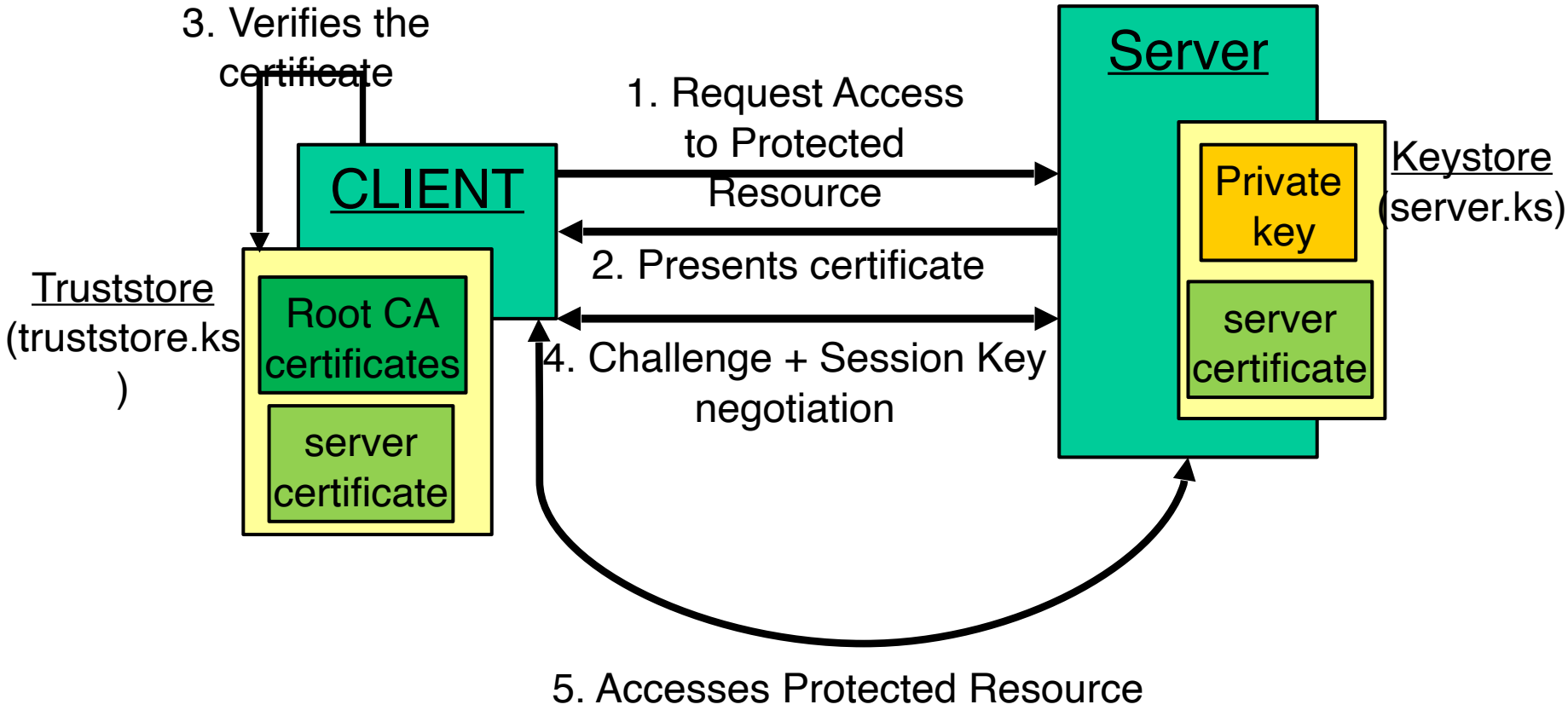


HTTPS (HIGH LEVEL OVERVIEW)



HTTPS (High Level Overview)

Naturally, the client might be optionally obligated to authenticate itself in between steps 3 and 4 (e.g., by presenting its own certificate using TLS, or by providing username/password at the application level)



GOALS

In the end of this lab you should be able to:

- Understand what is HTTPS and SSL/TLS
- **Know how to generate a keystore with server cryptographic keys**
- **Know how to generate a truststore with root certificates and the certificate for your server**
- Know how to develop a REST server using https in Java
- Know how to develop a gRPC server using TLS in Java
- Know how to modify your REST clients to use https
- Know how to modify your gRPC clients to use TLS

Generating keystores and TrustStores

From the previous explanation, you should have figured out that we need to create the keystore and truststore used by the servers and clients respectively.

- The authentication of the server in SSL/TLS uses public key certificates (https://en.wikipedia.org/wiki/Public_key_certificate)
- In the context of Java, both certificates and (private) keys are stored in a keystore.
- The keytool is the command in the java environment that allows to manipulate keystores, certificates and keys.

GENERATING THE SERVER KEYSTORE

CLIENT

Server

To generate the server keystore (which will include the server private key and certificate you should use the following command (in the root directory of your project):

```
keytool -ext SAN=dns:<server-name> -genkeypair -alias <server-name>
-keyalg RSA -validity 365 -keystore <keystore-filename> -storetype
pkcs12
```

GENERATING THE SERVER KEYSTORE

CLIENT

Server

To generate the server keystore (which will include the server private key and certificate you should use the following command (in the root directory of your project):

```
keytool -ext SAN=dns:users.domain -genkeypair -alias users.domain
-keyalg RSA -validity 365 -keystore users-domain-server.ks
-stor
```

For instance, assuming that the server is named **users.domain** and that it will run on a machine/container named **users.domain**

GENERATING THE SERVER KEYSTORE

CLIENT

Server

To generate the server keystore (which will include the server private key and certificate you should use the following command (in the root directory of your project):

```
keytool -ext SAN=dns:users.domain -genkeypair -alias users.domain
-keyalg RSA -validity 365 -keystore users-domain-server.ks
-stor
```

This command generates a public key certificate identified by name 'users.domain', that is valid for 365 days, that will be stored in a keystore in a file named 'users-domain-server.ks', being signed by its own key (self-signed certificate)

GENERATING THE SERVER KEYSTORE

CLIENT

Server

```
nmp@10-139-25-126 lab8 % keytool -ext SAN=dns:users.domain -genkeypair -alias users.domain -keyalg RSA -validity 365 -
keystore users-domain-server.ks -storetype pkcs12
Enter keystore password:
Re-enter new password:
Enter the distinguished name. Provide a single dot (.) to leave a sub-component empty or press ENTER to use the defaul
t value in braces.
What is your first and last name?
  [Unknown]:  users.domain
What is the name of your organizational unit?
  [Unknown]:
What is the name of your organization?
  [Unknown]:
What is the name of your City or Locality?
  [Unknown]:
What is the name of your State or Province?
  [Unknown]:
What is the two-letter country code for this unit?
  [Unknown]:
Is CN=users.domain, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown correct?
  [no]:  yes

Generating 3072-bit RSA key pair and self-signed certificate (SHA384withRSA) with a validity of 365 days
        for: CN=users.domain, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown
```

GENERATING THE SERVER KEYSTORE

CLIENT

Server

```
nmp@10-139-25-126 lab8 % keytool -ext SAN=dns:users.domain,dns:bank.domain,dns:alice.users.domain -keyalg RSA -validity 365
keystore users-domain-server.ks -storetype pkcs12
Enter keystore password:
Re-enter new password:
Enter the distinguished name. Provide a single dot
t value in braces.
What is your first and last name?
  [Unknown]:  users.domain
What is the name of your organizational unit?
  [Unknown]:
What is the name of your organization?
  [Unknown]:
What is the name of your City or Locality?
  [Unknown]:
What is the name of your State or Province?
  [Unknown]:
What is the two-letter country code for this unit?
  [Unknown]:
Is CN=users.domain, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown correct?
  [no]:  yes

Generating 3072-bit RSA key pair and self-signed certificate (SHA384withRSA) with a validity of 365 days
        for: CN=users.domain, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown
```

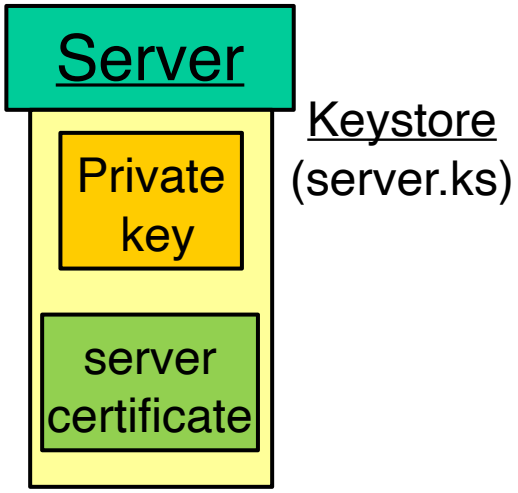
It will ask you for a password to protect the keystore. I have used: password

GENERATING THE SERVER KEYSTORE

CLIENT

```
nmp@10-139-25-126 lab8 % keytool -ext SAN=dns:users.domain -genkeypair -alias users.domain -keyalg RSA -validity 365 -
keystore users-domain-server.ks -storetype pkcs12
Enter keystore password:
Re-enter new password:
Enter the distinguished name. Provide a single dot (.) to leave a sub-component empty or press ENTER to use the defaul
t value in braces.
What is your first and last name?
  [Unknown]:  users.domain
What is the name of your organizational unit?
  [Unknown]:
What is the name of your organization?
  [Unknown]:
What is the name of your City or Locality?
  [Unknown]:
What is the name of your State or Province?
  [Unknown]:
What is the two-letter country code for this unit?
  [Unknown]:
Is CN=users.domain, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown correct?
  [no]:  yes

Generating 3072-bit RSA key pair and self-signed certificate (SHA384withRSA) with a validity of 365 days
        for: CN=users.domain, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown
```



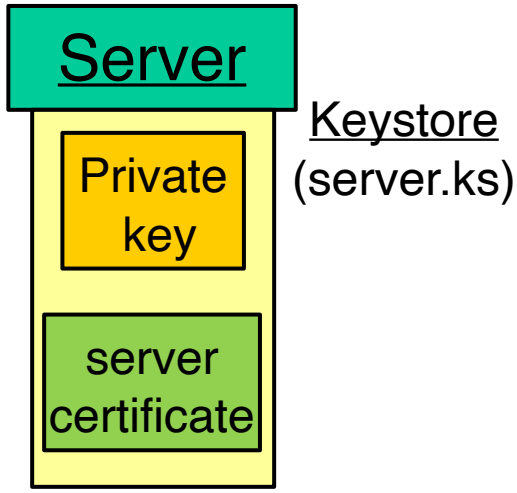
After executing this command successfully, you will have created the server keystore (in a file named: users-domain-*server.ks*) that includes within it the server private key and the server public key certificate.

TRUST IN THE WEB

CLIENT

```
nmp@10-139-25-126 lab8 % keytool -ext SAN=dns:users.domain -genkeypair -alias users.domain -keyalg RSA -validity 365 -
keystore users-domain-server.ks -storetype pkcs12
Enter keystore password:
Re-enter new password:
Enter the distinguished name. Provide a single dot (.) to leave a sub-component empty or press ENTER to use the defaul
t value in braces.
What is your first and last name?
  [Unknown]:  users.domain
What is the name of your organizational unit?
  [Unknown]:
What is the name of your organization?
  [Unknown]:
What is the name of your City or Locality?
  [Unknown]:
What is the name of your State or Province?
  [Unknown]:
What is the two-letter country code for this unit?
  [Unknown]:
Is CN=users.domain, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown correct?
  [no]:  yes

Generating 3072-bit RSA key pair and self-signed certificate (SHA384withRSA) with a validity of 365 days
        for: CN=users.domain, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown
```



Web server have certificates signed by “trusted” certification authorities. The developer of an application would export the public key/certificate, have it signed by the certification authority and add it to the keystore again. The TLS protocol would exchange the certificates and parties would verify their validity. In our system, we will add certificate to the client Truststore instead.

GENERATING THE CLIENT TRUSTSTORE



We now need to generate the client truststore. Notice that a truststore is similar to a keystore with the exception that it only stores public key certificates belonging to other entities.

Since we want to allow our clients to interact with any server in the world, its truststore will have to contain the certificates of all Root Certification Authorities (such as VerySign or LetsEncrypt).

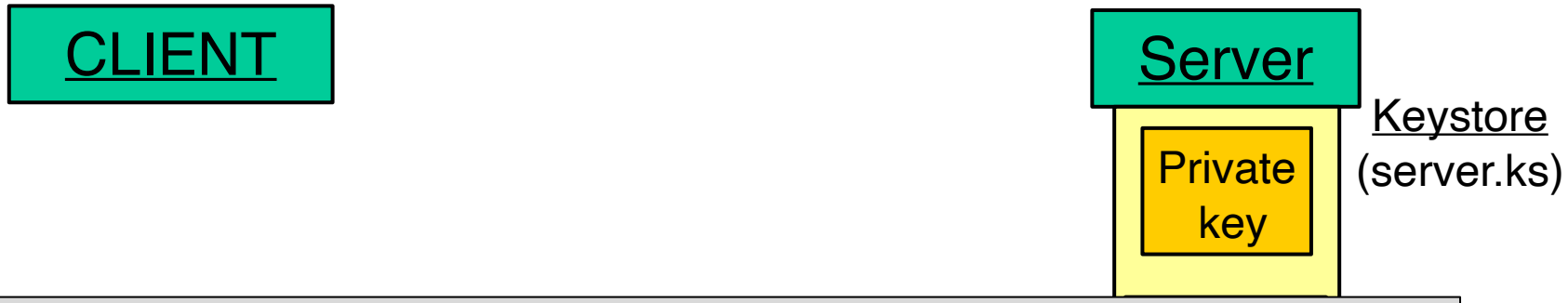
GENERATING THE CLIENT TRUSTSTORE



To do that we will initialize the client truststore with a copy of the Java environment **cacerts**, the default truststore that contains all Root CA certificates with the command:

```
cp /Library/Java/JavaVirtualMachines/jdk-17.jdk/Contents/
Home/lib/security/cacerts truststore.ks
```

GENERATING THE CLIENT TRUSTSTORE



CACERTS

The default Java truststore cacerts contains just a list of root CA certificates.

These are certificates issued by "Certification Authorities" that Java trusts implicitly.

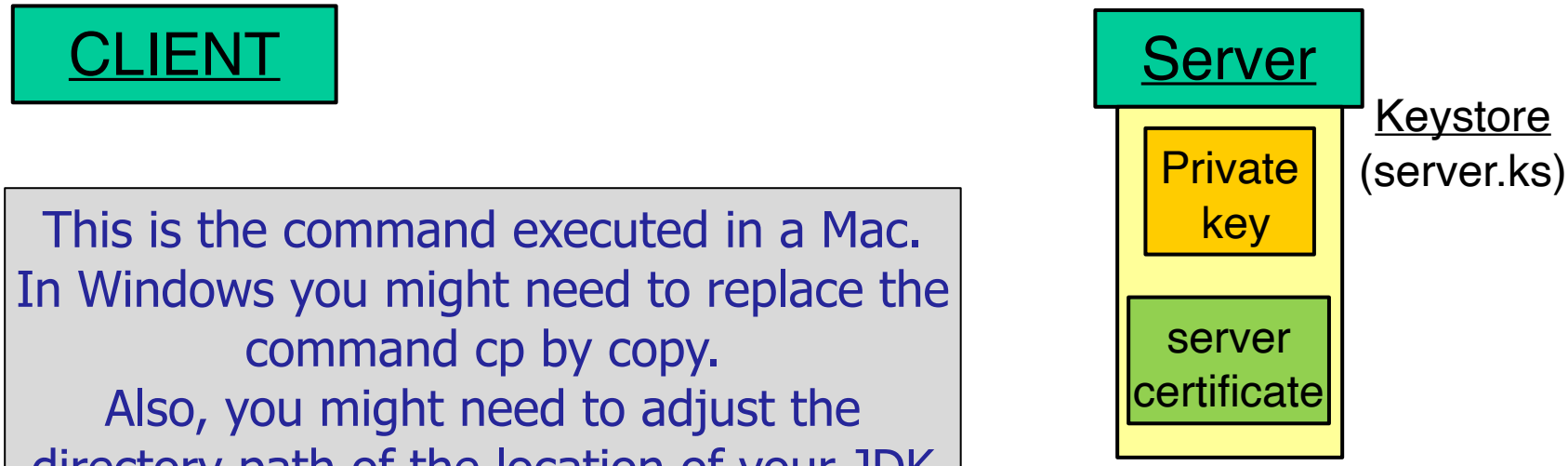
They are stored in a file named cacerts included in every JDK and JRE distribution.

The default cacertspassword is changeit.

To do th e Java
environ ot CA
certifica

```
cp /Lib                    s/
Home/li
```

GENERATING THE CLIENT TRUSTSTORE



This is the command executed in a Mac.
In Windows you might need to replace the command cp by copy.
Also, you might need to adjust the directory path of the location of your JDK installation.

To do that we will initialize the client truststore with a copy of the Java environment cacerts, the default truststore that contains all Root CA certificates with the command:

```
cp /Library/Java/JavaVirtualMachines/jdk-17.jdk/Contents/Home/lib/security/cacerts truststore.ks
```

GENERATING THE CLIENT TRUSTSTORE



After executing this command successfully, you will have created the truststore for the client (in a file named: truststore.ks) containing all Root CA certificates. Since this a copy of the Java cacerts, it is protected by the password: changeit

```
cp /Library/Java/JavaVirtualMachines/jdk-17.jdk/Contents/Home/lib/security/cacerts
truststore.ks
```

EXPORTING THE SERVER CERTIFICATE



We still need to insert the server certificate in the client truststore.

However, to do that we will need first to export the server certificate from the server keystore to a temporary file.

EXPORTING THE SERVER CERTIFICATE



To export the server certificate, we use again keytool. You should run the following command:

keytool -exportcert -alias <server-name> -keystore <keystore-file> -file <certificate-file>

EXPORTING THE SERVER CERTIFICATE



To export the server certificate, we use again keytool. You should run the following command:

keytool -exportcert -alias users.domain -keystore users-domain-server.ks -file users-domain.cert

This command exports the certificate of entity named 'users.domain' from the keystore file 'users-domain-server.ks' into a temporary file named 'users-domain.cert'

EXPORTING THE SERVER CERTIFICATE




```
nmp@10-139-25-126 lab8 % keytool -exportcert -alias users.domain -keystore users-domain-server.ks -file users-domain.cert

Enter keystore password:
Certificate stored in file <users-domain.cert>
```

EXPORTING THE SERVER CERTIFICATE

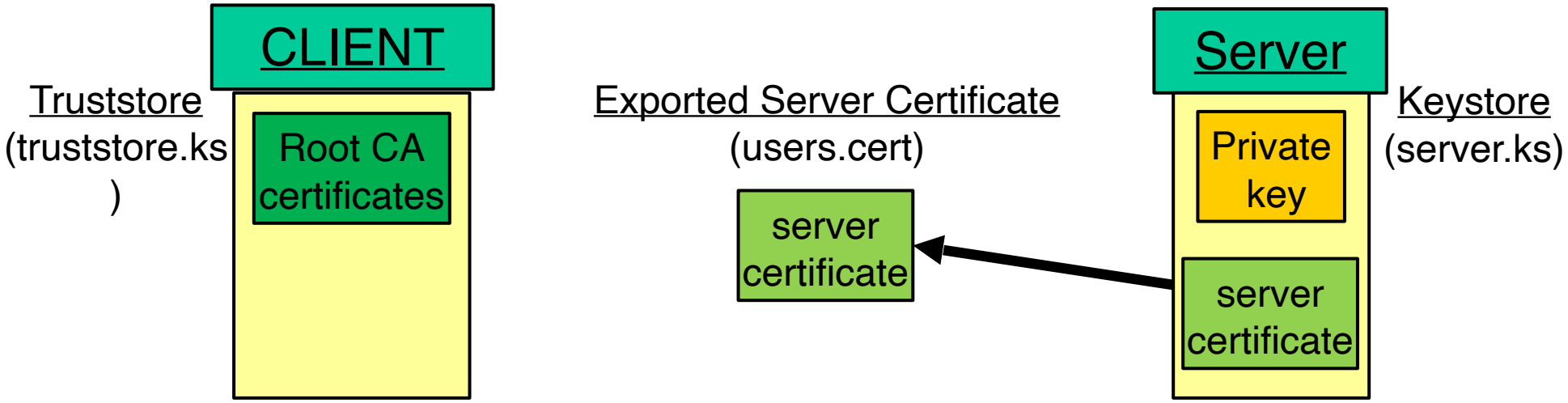


It will ask you for the password that protects the keystore file.

```
nmp@10-139-25-126 lab8 % keytool -exportcert -alias users.domain -keystore users-domain-server.ks -file users-domain.cert

Enter keystore password:
Certificate stored in file <users-domain.cert>
```

EXPORTING THE SERVER CERTIFICATE

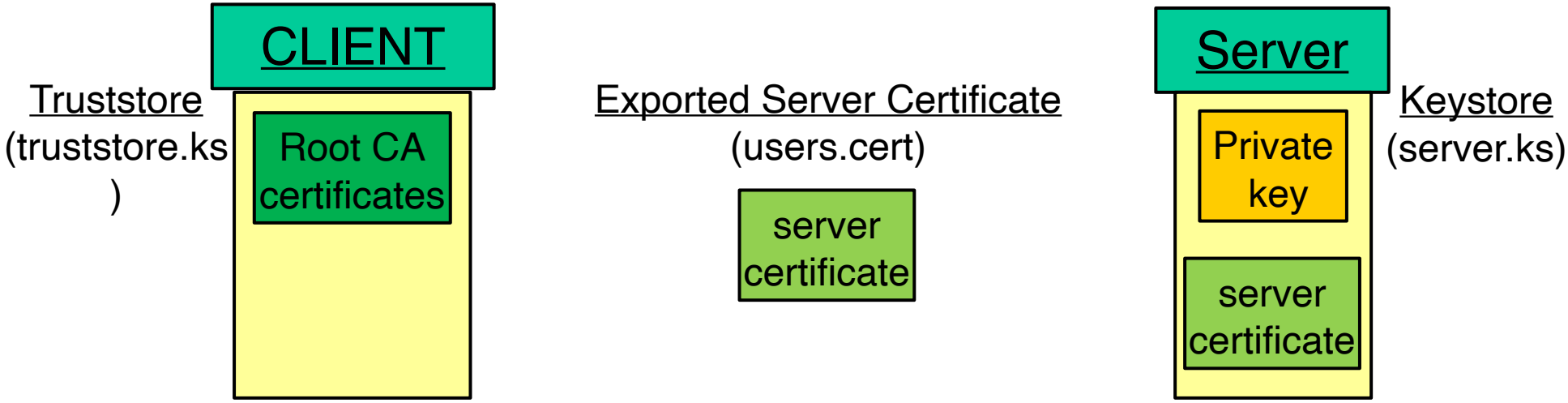


After executing this command successfully, you will have created a temporary file (users-domain.cert) with the server certificate

```
nmp@10-139-25-126 lab8 % keytool -exportcert -alias users.domain -keystore users-domain-server.ks -file users-domain.cert

Enter keystore password:
Certificate stored in file <users-domain.cert>
```

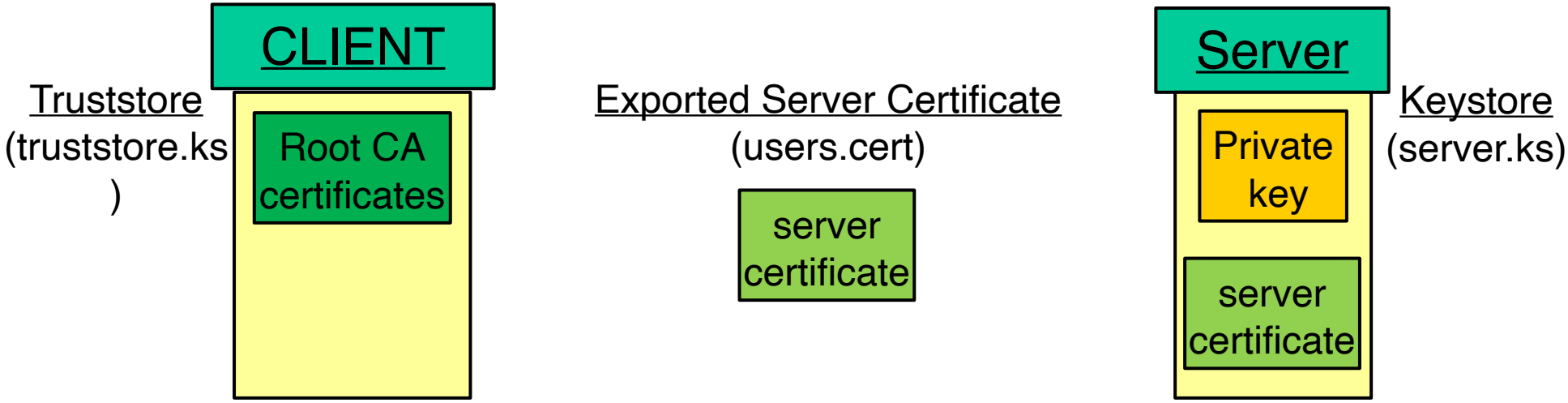
IMPORTING THE SERVER CERTIFICATE



Now that we have exported the server certificate, we just need to add it to the client truststore. Again, we will use keytool to do this:

keytool -importcert -file <certificate-file> -alias <server-name> -keystore <keystore-file>

IMPORTING THE SERVER CERTIFICATE



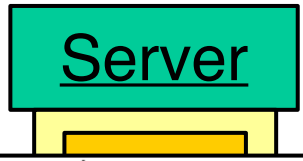
Now that we have exported the server certificate, we just need to add it to the client truststore. Again, we will use keytool to do this:

keytool -importcert -file users-domain.cert -alias users.domain -keystore truststore.ks

This command imports the certificate of entity named 'users.domain', located in file 'users-domain.cert', into the truststore (i.e., a keystore) named 'truststore.ks'

IMPORTING THE SERVER CERTIFICATE





Truststore

Exported Server Certificate

Keystore

```
nmp@10-139-25-126 lab8 % keytool -importcert -file users-domain.cert -alias users.domain -keystore truststore.ks

Enter keystore password:
Owner: CN=users.domain, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown
Issuer: CN=users.domain, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown
Serial number: 5138604ccfadd032
Valid from: Wed Apr 08 10:41:50 WEST 2026 until: Thu Apr 08 10:41:50 WEST 2027
Certificate fingerprints:
         SHA1: 24:0C:34:8C:74:C3:DA:91:EA:AB:36:A2:64:69:7C:68:68:CA:37:B2
         SHA256: F8:4B:8D:54:0D:99:40:2F:C6:84:E0:C7:2A:82:5B:84:3A:4C:96:A8:57:59:D0:67:7C:E7:55:68:9A:58:A3:17
Signature algorithm name: SHA384withRSA
Subject Public Key Algorithm: 3072-bit RSA key
Version: 3

Extensions:

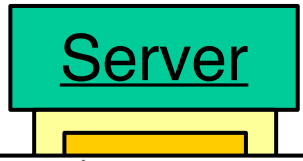
#1: ObjectId: 2.5.29.17 Criticality=false
SubjectAlternativeName [
  DNSName: users.domain
]

#2: ObjectId: 2.5.29.14 Criticality=false
SubjectKeyIdentifier [
KeyIdentifier [
0000: 62 DC 3B 94 AF 5D 9C B4   74 73 8F 94 09 88 ED FC  b.;..]..ts......
0010: 95 A4 3D 8E                                        ..=.
]
]

Trust this certificate? [no]:  yes
Certificate was added to keystore
```

IMPORTING THE SERVER CERTIFICATE





Truststore

Exported Server Certificate

Keystore

```
nmp@10-139-25-126 lab8 % keytool -importcert -file users-domain.cert -alias users.domain -keystore truststore.ks

Enter keystore password:
Owner: CN=users.domain, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown
Issuer: CN=users.domain, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown
Serial number: 5138604ccfadd032
Valid from: Wed Apr 08 10:41:50 WEST 2026 until: Thu Apr 08 10:41:50 WEST 2027
Certificate fingerprints:
         SHA1: 24:0C:34:8C:74:C3:DA:91:EA:AB:36:A2:64:69:7C:68:68:CA:37:B2
         SHA256: F8:4B:8D:54:0D:99:40:2F:C6:24:E0:C7:2A:82:5B:84:3A:4C:96:A8:57:59:D0:67:7C:E7:55:68:9A:58:A3:17
Signature algorithm name: SHA384withRSA
Subject Public Key Algorithm: 3072-bit RSA key
Version: 3

Extensions:

#1: ObjectId: 2.5.29.17 Criticality=false
SubjectAlternativeName [
  DNSName: users.domain
]

#2: ObjectId: 2.5.29.14 Criticality=false
SubjectKeyIdentifier [
KeyIdentifier [
0000: 62 DC 3B 94 AF 5D 9C B4   74 73 8F 94 09 88 ED FC  b.;..]..ts......
0010: 95 A4 3D 8E                                        ..=.
]
]

Trust this certificate? [no]:  yes
Certificate was added to keystore
```

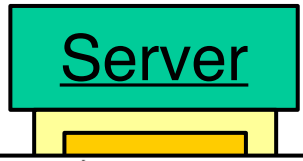
It will ask you for the password that protects the truststore file.
As said before this is the password of the original cacerts file: changeit

IMPORTING THE SERVER CERTIFICATE



Truststore

Exported Server Certificate



Keystore

```
nmp@10-139-25-126 lab8 % keytool -importcert -file users-domain.cert -alias users.domain -keystore truststore.ks

Enter keystore password:
Owner: CN=users.domain, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown
Issuer: CN=users.domain, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown
Serial number: 5138604ccfadd032
Valid from: Wed Apr 08 10:41:50 WEST 2026 until: Thu Apr 08 10:41:50 WEST 2027
Certificate fingerprints:
         SHA1: 24:0C:34:8C:74:C3:DA:91:EA:AB:36:A2:64:69:7C:68:68:CA:37:B2
         SHA256: F8:4B:8D:54:0D:99:40:2F:C6:84:E0:C7:2A:82:5B:84:3A:4C:96:A8:57:59:D0:67:7C:E7:55:68:9A:58:A3:17
Signature algorithm name: SHA384withRSA
Subject Public Key Algorithm: 3072-bit RSA key
Version: 3

Extensions:

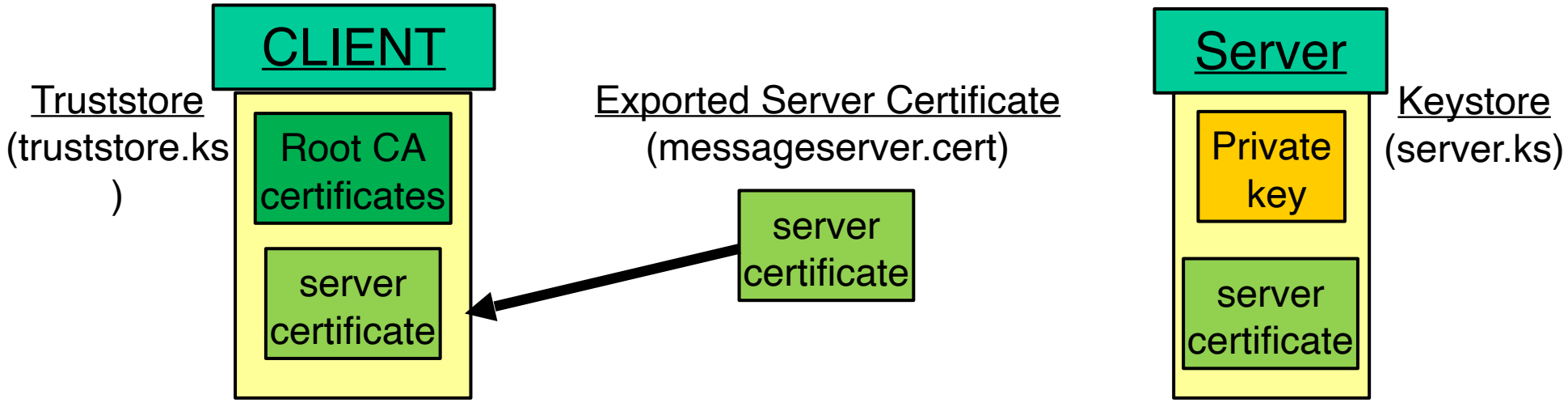
#1: ObjectId: 2.5.29.17 Criticality=false
SubjectAlternativeName [
  DNSName: users.domain
]

#2: ObjectId: 2.5.29.14 Criticality=false
SubjectKeyIdentifier [
KeyIdentifier [
0000: 62 DC 3B 94 AF 5D 9C B4   74 73 8F 94 09 88 E
0010: 95 A4 3D 8E
]
]

Trust this certificate? [no]:  yes
Certificate was added to keystore
```

After showing you the fingerprint of the certificate that is being imported as well as the identity of the owner of the certificate you will have to confirm that you want to trust this certificate.

Importing the Server Certificate




```
nmp@10-139-25-126 lab8 % keytool -importcert -file users-domain.cert -alias users.domain -keystore truststore.ks

Enter keystore password:
Owner: CN=users.domain, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown
Issuer: CN=users.domain, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown
Serial number: 5138604ccfadd032
Valid from: Wed Apr 08 10:41:50 WEST 2026 until: Thu Apr 08 10:41:50 WEST 2027
Certificate fingerprints:
         SHA1: 24:0C:34:8C:74:C3:DA:91:EA:AB:36:A2:64:69:7C:68:68:CA:37:B2
         SHA256: F8:4B:8D:54:0D:99:40:2F:C6:84:E0:C7:2A:82:5B:84:3A:4C:96:A8:57:59:D0:67:7C:E7:55:68:9A:58:A3:17
Signature algorithm name: SHA384withRSA
Subject Public Key Algorithm: 3072-bit RSA key
Version: 3

Extensions:

#1: ObjectId: 2.5.29.17 Criticality=false
SubjectAlternativeName [
  DNSName: users.domain
]

#2: ObjectId: 2.5.29.14 Criticality=false
SubjectKeyIdentifier [
KeyIdentifier [
0000: 62 DC 3B 94 AF 5D 9C B4   74 73 8F 94 09 88 ED FC  b.;..]..ts......
0010: 95 A4 3D 8E                                        ..=.
]
]

Trust this certificate? [no]:  yes
Certificate was added to keystore
```

After executing this command successfully, you will have added the server certificate to the client truststore.

GOALS

In the end of this lab you should be able to:

- Understand what is HTTPS and SSL/TLS
- Know how to generate a keystore with server cryptographic keys
- Know how to generate a truststore with root certificates and the certificate for your server
- **Know how to develop a REST server using https in Java**
- **Know how to develop a gRPC server using TLS in Java**
- Know how to modify your REST clients to use https
- Know how to modify your gRPC clients to use TLS

REST SERVER AND HTTPS

Class containing the main of the Rest Server.

```
public abstract class AbstractRestServer {
    private static final String SERVER_BASE_URI = "https://%s:%s/rest";
    static {
        System.setProperty("java.net.preferIPv4Stack", "true");
        System.setProperty("java.util.logging.SimpleFormatter.format", "%4$s: %5$s\n");
    }

    final protected int port;
    final protected Logger Log;
    final protected String service;
    final protected String serverURI;
    protected AbstractRestServer(Logger log, String service, int port) throws UnknownHostException {
        this.Log = log;
        this.port = port;
        this.service = service;
        this.serverURI = SERVER_BASE_URI.formatted(InetAddress.getLocalHost().getHostName(), port);
    }

    protected void start() {
        try {
            ResourceConfig config = new ResourceConfig();
            registerResources( config);
            var uri = URI.create("https://0.0.0.0:%s/rest".formatted(port));
            System.out.println( uri);
            JdkHttpServerFactory.createHttpServer( uri, config, javax.net.ssl.SSLContext.getDefault());
            String domain = "mydomain"; // TODO: Replace with proper domain...
            Discovery.announce("%s@%s".formatted(service, domain), serverURI);
            Log.info(String.format("%s Server ready @ %s\n", service, serverURI));
        } catch (Exception e) {
            Log.severe("Failed to start server: %s".formatted(e.getMessage()));
        }
    }
    abstract void registerResources( ResourceConfig config);
}
```

REST SERVER AND HTTPS

Class containing the main of the Rest Server.

```
public abstract class AbstractRestServer {
    private static final String SERVER_BASE_URI = "https://%s:%s/rest";
    static {
        System.setProperty("java.net.preferIPv4Stack", "true");
        System.setProperty("java.util.logging.SimpleFormatter.format", "%4$s: %5$s\n");
    }

    final protected int port;
    final protected Logger Log;
    final protected String service;
    final protected String serverURI;
    protected AbstractRestServer(Logger log, String service, int port) throws UnknownHostException {
        this.Log = log;
        this.port = port;
        this.service = service;
        this.serverURI = SERVER_BASE_URI.formatted(InetAddress.getLocalHost().getHostName(), port);
    }

    protected void start() {
        try {
            ResourceConfig config = new ResourceConfig();
            registerResources( config);
            var uri = URI.create("https://0.0.0.0:%s/rest".formatted(port));
            System.out.println( uri);
            JdkHttpServerFactory.createHttpServer( uri, config, javax.net.ssl.SSLContext.getDefault());
            String domain = "mydomain"; // TODO: Replace with proper domain...
            Discovery.announce("%s@%s".formatted(service, domain), serverURI);
            Log.info(String.format("%s Server ready @ %s\n", service, serverURI));
        } catch (Exception e) {
            Log.severe("Failed to start server: %s".formatted(e.getMessage()));
        }
    }
    abstract void registerResources( ResourceConfig config);
}
```

You need to change the URL of the server to have https:// instead of http://

REST SERVER AND HTTPS

Class containing the main of the Rest Server.

```
public abstract class AbstractRestServer {
    private static final String SERVER_BASE_URI = "https://%s:%s/rest";
    static {
        System.setProperty("java.net.preferIPv4Stack", "true");
        System.setProperty("java.util.logging.SimpleFormatter.format", "%4$s: %5$s\n");
    }

    final protected int port;
    final protected Logger Log;
    final protected String service;
    final protected String serverURI;
    protected AbstractRestServer(Logger log, String service, int port) throws UnknownHostException {
        this.Log = log;
        this.port = port;
        this.service = service;
        this.serverURI = SERVER_BASE_URI.formatted(InetAddress.getLocalHost().getHostName(), port);
    }

    protected void start() {
        try {
            ResourceConfig config = new ResourceConfig()
            registerResources( config);
            var uri = URI.create("https://0.0.0.0:%s/rest
            System.out.println( uri);
            JdkHttpServerFactory.createHttpServer( uri, c
            String domain = "mydomain"; // TODO: Replace
            Discovery.announce("%s@%s".formatted(service
            Log.info(String.format("%s Server ready @ %s
        } catch (Exception e) {
            Log.severe("Failed to start server: %s".formatted(e.getMessage()));
        }
    }
    abstract void registerResources( ResourceConfig config);
}
```

The URL that clients use to contact the server can no longer have the IP address of the server and instead should have the hostname (that should be the <server-name> use to generate the server private key and certificate.

REST SERVER AND HTTPS

Class containing the main of the Rest Server.

The way that you start the HTTP server must be modified to include the default SSLContext (that is responsible for managing keystore and truststore)

```
public abstract class AbstractRestServer {
    private static final String SERVER_BASE_URI = "https://%s:%s/rest";
    static {
        System.setProperty("java.net.preferIPv4Stack", "true");
        System.setProperty("java.util.logging.SimpleFormatter.format", "%4$s: %5$s\n");
    }

    final protected int port;
    final protected Logger Log;
    final protected String service;
    final protected String serverURI;
    protected AbstractRestServer(Logger log, String s
        this.Log = log;
        this.port = port;
        this.service = service;
        this.serverURI = SERVER_BASE_URI.formatted(InetAddress.getLocalHost().getHostName(), port);
    }

    protected void start() {
        try {
            ResourceConfig config = new ResourceConfig();
            registerResources( config);
            var uri = URI.create("https://0.0.0.0:%s/rest".formatted(port));
            System.out.println( uri);
            JdkHttpServerFactory.createHttpServer( uri, config, javax.net.ssl.SSLContext.getDefault());
            String domain = "mydomain" ; // TODO: Replace with proper domain...
            Discovery.announce("%s@%s".formatted(service, domain), serverURI);
            Log.info(String.format("%s Server ready @ %s\n", service, serverURI));
        } catch (Exception e) {
            Log.severe("Failed to start server: %s".formatted(e.getMessage()));
        }
    }
    abstract void registerResources( ResourceConfig config);
}
```

REST SERVER AND HTTPS

Class containing the main of the Rest Server.

```
public abstract class AbstractRestServer {
    private static final String SERVER_BASE_URI = "https://%s:%s/rest";
    static {
        System.setProperty("java.net.preferIPv4Stack", "true");
        System.setProperty("java.util.logging.SimpleFormatter.format", "%4$s: %5$s\n");
    }

    final protected int port;
    final protected Logger Log;
    final protected String service;
    final protected String serverURI;
    protected AbstractRestServer(Logger log, String service, int port) throws UnknownHostException {
        this.Log = log;
        this.port = port;
        this.service = service;
        this.serverURI = SERVER_BASE_URI.formatted(InetAddress.getLocalHost().getHostName(), port);
    }

    protected void start() {
        try {
            ResourceConfig config = new ResourceConfig();
            registerResources( config);
            var uri = URI.create("https://0.0.0.0:%s/rest".formatted(port));
            System.out.println( uri);
            JdkHttpServerFactory.createHttpServer( uri, config, javax.net.ssl.SSLContext.getDefault());
            String domain = "mydomain"; // TODO: Replace with proper domain...
            Discovery.announce("%s@%s".formatted(service, domain), serverURI);
            Log.info(String.format("%s Server ready @ %s\n", service, serverURI));
        } catch (Exception e) {
            Log.severe("Failed to start server: %s".formatted(e.getMessage()));
        }
    }
    abstract void registerResources( ResourceConfig config);
}
```

And these covers all changes to the server side in REST.

gRPC Server and TLS

Class containing the main of the gRPC Server.

```
public class GrpcUsersServer {
    public static final int PORT = 9000;

    private static final String GRPC_CTX = "/grpc";
    private static final String SERVER_BASE_URI = "grpc://%s:%s%s";
    private static Logger Log = Logger.getLogger(GrpcUsersServer.class.getName());

    public static void main(String[] args) throws Exception {
        String keyStoreFilename = System.getProperty("javax.net.ssl.keyStore");
        String keyStorePassword = System.getProperty("javax.net.ssl.keyStorePassword");
        KeyStore keystore = KeyStore.getInstance(KeyStore.getDefaultType());
        try(FileInputStream input = new FileInputStream(keyStoreFilename)) {
            keystore.load(input, keyStorePassword.toCharArray());
        }
        KeyManagerFactory keyManagerFactory = KeyManagerFactory.getInstance(
            KeyManagerFactory.getDefaultAlgorithm());
        keyManagerFactory.init(keystore, keyStorePassword.toCharArray());

        SslContext context = GrpcSslContexts.configure(
            SslContextBuilder.forServer(keyManagerFactory)
            ).build();
        GrpcUsersController stub = new GrpcUsersController();

        Server server = NettyServerBuilder.forPort(PORT)
            .addService(stub).sslContext(context).build();

        String serverURI = String.format(SERVER_BASE_URI, InetAddress.getLocalHost().getHostName(), PORT, GRPC_CTX);

        Log.info(String.format("Users gRPC Server ready @ %s\n", serverURI));
        server.start().awaitTermination();
    }
}
```

gRPC Server and TLS

Class containing the main of the gRPC Server.

```
public class GrpcUsersServer {
    public static final int PORT = 9000;

    private static final String GRPC_CTX = "/grpc";
    private static final String SERVER_BASE_URI = "grpc://%s:%s%s";
    private static Logger Log = Logger.getLogger(GrpcUsersServer.class.getName());

    public static void main(String[] args) throws Exc
        String keyStoreFilename = System.getProperty(
        String keyStorePassword = System.getProperty(
        KeyStore keystore = KeyStore.getInstance(KeyS
        try(FileInputStream input = new FileInputStre
            keystore.load(input, keyStorePassword.toC
        }
        KeyManagerFactory keyManagerFactory = KeyMana
            KeyManagerFactory.getDefaultAlgorithm());
        keyManagerFactory.init(keystore, keyStorePass

        SslContext context = GrpcSslContexts.configur
            SslContextBuilder.forServer(keyManagerFac
            ).build();
        GrpcUsersController stub = new GrpcUsersContr

        Server server = NettyServerBuilder.forPort(PO
            .addService(stub).sslContext(context).bui

        String serverURI = String.format(SERVER_BASE_URI, InetAddress.getLocalHost().getHostName(), PORT, GRPC_CTX);

        Log.info(String.format("Users gRPC Server ready @ %s\n", serverURI));
        server.start().awaitTermination();
    }
}
```

By default, gRPC will use keys and certificates in PEM format (https://en.wikipedia.org/wiki/Privacy-Enhanced_Mail).

It is possible to convert pkcs12 keys and certificates to PEM using the openssl (https://www.openssl.org/) command line tool.

Instead, we are going to take advantage of the integration of gRPC with the netty (https://netty.io/) communication library that allows to use pkc12 keys.

gRPC Server and TLS

Class containing the main of the gRPC Server.

```
public class GrpcUsersServer {
    public static final int PORT = 9000;

    private static final String GRPC_CTX = "/grpc";
    private static final String SERVER_BASE_URI = "grpc://%s:%s%s";
    private static Logger Log = Logger.getLogger(GrpcUsersServer.class.getName());

    public static void main(String[] args) throws Exception {
        String keyStoreFilename = System.getProperty("javax.net.ssl.keyStore");
        String keyStorePassword = System.getProperty("javax.net.ssl.keyStorePassword");
        KeyStore keystore = KeyStore.getInstance(KeyStore.getDefaultType());
        try(FileInputStream input = new FileInputStream(keyStoreFilename)) {
            keystore.load(input, keyStorePassword.toCharArray());
        }
        KeyManagerFactory keyManagerFactory = KeyManagerFactory.getInstance(
            KeyManagerFactory.getDefaultAlgorithm());
        keyManagerFactory.init(keystore, keyStorePassword.toCharArray());

        SslContext context = GrpcSslContexts.configure(
            SslContextBuilder.forServer(keyManagerFacto
            ).build();
        GrpcUsersController stub = new GrpcUsersControl

        Server server = NettyServerBuilder.forPort(PORT
            .addService(stub).sslContext(context).build

        String serverURI = String.format(SERVER_BASE_UR

        Log.info(String.format("Users gRPC Server ready @ %s\n", serverURI));
        server.start().awaitTermination();
    }
}
```

We will start by accessing the values of the default JVM properties (we will see how we configure these later) regarding both the filename that contains the server keystore and the password that protects that keystore.

gRPC Server and TLS

Class containing the main of the gRPC Server.

```
public class GrpcUsersServer {
    public static final int PORT = 9000;

    private static final String GRPC_CTX = "/grpc";
    private static final String SERVER_BASE_URI = "grpc://%s:%s%s";
    private static Logger Log = Logger.getLogger(GrpcUsersServer.class.getName());

    public static void main(String[] args) throws Exception {
        String keyStoreFilename = System.getProperty("javax.net.ssl.keyStore");
        String keyStorePassword = System.getProperty("javax.net.ssl.keyStorePassword");
        KeyStore keystore = KeyStore.getInstance(KeyStore.getDefaultType());
        try(FileInputStream input = new FileInputStream(keyStoreFilename)) {
            keystore.load(input, keyStorePassword.toCharArray());
        }
        KeyManagerFactory keyManagerFactory = KeyManagerFactory.getInstance(
            KeyManagerFactory.getDefaultAlgorithm());
        keyManagerFactory.init(keystore, keyStorePassword.toCharArray());

        SslContext context = GrpcSslContexts.configure(
            SslContextBuilder.forServer(keyManagerFactory)
            ).build();
        GrpcUsersController stub = new GrpcUsersController();

        Server server = NettyServerBuilder.forPort(PORT)
            .addService(stub).sslContext(context).build();

        String serverURI = String.format(SERVER_BASE_URI

        Log.info(String.format("Users gRPC Server ready
        server.start().awaitTermination();
    }
}
```

Then we obtain an instance of a keystore (initially empty) and load it with the cryptographic information stored in the server keystore. To do that we open a FileInputStream to the keystore file and load its contents providing the key that protects the file.

gRPC Server and TLS

Class containing the main of the gRPC Server.

```
public class GrpcUsersServer {
    public static final int PORT = 9000;

    private static final String GRPC_CTX = "/grpc";
    private static final String SERVER_BASE_URI = "grpc://%s:%s%s";
    private static Logger Log = Logger.getLogger(GrpcUsersServer.class.getName());

    public static void main(String[] args) throws Exception {
        String keyStoreFilename = System.getProperty("javax.net.ssl.keyStore");
        String keyStorePassword = System.getProperty("javax.net.ssl.keyStorePassword");
        KeyStore keystore = KeyStore.getInstance(KeyStore.getDefaultType());
        try(FileInputStream input = new FileInputStream(keyStoreFilename)) {
            keystore.load(input, keyStorePassword.toCharArray());
        }
        KeyManagerFactory keyManagerFactory = KeyManagerFactory.getInstance(
            KeyManagerFactory.getDefaultAlgorithm());
        keyManagerFactory.init(keystore, keyStorePassword.toCharArray());

        SslContext context = GrpcSslContexts.configure(
            SslContextBuilder.forServer(keyManagerFactory)
            ).build();
        GrpcUsersController stub = new GrpcUsersController();

        Server server = NettyServerBuilder.forPort(PORT)
            .addService(stub).sslContext(context).build(

        String serverURI = String.format(SERVER_BASE_URI

        Log.info(String.format("Users gRPC Server ready
        server.start().awaitTermination();
    }
}
```

Next, we need a KeyManagerFactory that can be generated using its factory and using the default cryptographic algorithms.
We then load into this factory the keystore that we have previously prepared (again providing the password that protects that keystore).

gRPC Server and TLS

Class containing the main of the gRPC Server.

We can now initialize an SslContext that will leverage the KeyManagerFactory to be able to access the private key and public key certificate of the server.

```
public class GrpcUsersServer {
    public static final int PORT = 9000;

    private static final String GRPC_CTX = "/grpc";
    private static final String SERVER_BASE_URI = "grpc://%s:%s%s";
    private static Logger Log = Logger.getLogger(Grp

    public static void main(String[] args) throws Exc
        String keyStoreFilename = System.getProperty(
        String keyStorePassword = System.getProperty(
        KeyStore keystore = KeyStore.getInstance(Key
        try(FileInputStream input = new FileInputStream(keyStoreFilename)) {
            keystore.load(input, keyStorePassword.toCharArray());
        }
        KeyManagerFactory keyManagerFactory = KeyManagerFactory.getInstance(
            KeyManagerFactory.getDefaultAlgorithm());
        keyManagerFactory.init(keystore, keyStorePassword.toCharArray());

        SslContext context = GrpcSslContexts.configure(
            SslContextBuilder.forServer(keyManagerFactory)
            ).build();
        GrpcUserServiceController stub = new GrpcUserServiceController();

        Server server = NettyServerBuilder.forPort(PORT)
            .addService(stub).sslContext(context).build();

        String serverURI = String.format(SERVER_BASE_URI, InetAddress.getLocalHost().getHostName(), PORT, GRPC_CTX);

        Log.info(String.format("Users gRPC Server ready @ %s\n", serverURI));
        server.start().awaitTermination();
    }
}
```

gRPC Server and TLS

Class containing the main of the gRPC Server.

We can now use the NettyServerBuilder (instead of the Grpc class as we did in the first project) to create an instance of Server, providing the port of the server, the instance of the gRPC server stub, and the SSL Context that we have prepared.

```
public class GrpcUsersServer {
    public static final int PORT = 9000;

    private static final String GRPC_CTX = "/grpc";
    private static final String SERVER_BASE_URI = "grpc://%s:%s%s";
    private static final Logger Log = Logger.getLogger(Grp

    public static void main(String[] args) throws Exc
        String keyStoreFilename = System.getProperty(
        String keyStorePassword = System.getProperty(
        KeyStore keystore = KeyStore.getInstance(Key
        try(FileInputStream input = new FileInputStre
            keystore.load(input, keyStorePassword.toC
        }
        KeyManagerFactory keyManagerFactory = KeyManagerFactory.getInstance(
            KeyManagerFactory.getDefaultAlgorithm());
        keyManagerFactory.init(keystore, keyStorePassword.toCharArray());

        SslContext context = GrpcSslContexts.configure(
            SslContextBuilder.forServer(keyManagerFactory)
            ).build();
        GrpcUsersController stub = new GrpcUsersController();

        Server server = NettyServerBuilder.forPort(PORT)
            .addService(stub).sslContext(context).build();

        String serverURI = String.format(SERVER_BASE_URI, InetAddress.getLocalHost().getHostName(), PORT, GRPC_CTX);

        Log.info(String.format("Users gRPC Server ready @ %s\n", serverURI));
        server.start().awaitTermination();
    }
}
```

gRPC Server and TLS

Class containing the main of the gRPC Server.

```
public class GrpcUsersServer {
    public static final int PORT = 9000;

    private static final String GRPC_CTX = "/grpc";
    private static final String SERVER_BASE_URI = "grpc://%s:%s%s";
    private static Logger Log = Logger.getLogger(Grp

    public static void main(String[] args) throws Exc
        String keyStoreFilename = System.getProperty(
        String keyStorePassword = System.getProperty(
        KeyStore keystore = KeyStore.getInstance(KeyS
        try(FileInputStream input = new FileInputStre
            keystore.load(input, keyStorePassword.toC
        }
        KeyManagerFactory keyManagerFactory = KeyMana
            KeyManagerFactory.getDefaultAlgorithm());
        keyManagerFactory.init(keystore, keyStorePassword.toCharArray());

        SslContext context = GrpcSslContexts.configure(
            SslContextBuilder.forServer(keyManagerFactory)
            ).build();
        GrpcUsersController stub = new GrpcUsersController();

        Server server = NettyServerBuilder.forPort(PORT)
            .addService(stub).sslContext(context).build();

        String serverURI = String.format(SERVER_BASE_URI, InetAddress.getLocalHost().getHostName(), PORT, GRPC_CTX);

        Log.info(String.format("Users gRPC Server ready @ %s\n", serverURI));
        server.start().awaitTermination();
    }
}
```

Similar to what we did in the REST server, the URL that the server announces for clients to connect to them can no longer have an IP address, instead it has to has the hostname of the machine where the server is running (and for which the private key and certificate were generated).

gRPC Server and TLS

Class containing the main of the gRPC Server.

```
public class GrpcUsersServer {
    public static final int PORT = 9000;

    private static final String GRPC_CTX = "/grpc";
    private static final String SERVER_BASE_URI = "grpc://%s:%s%s";
    private static Logger Log = Logger.getLogger(GrpcUsersServer.class.getName());

    public static void main(String[] args) throws Exception {
        String keyStoreFilename = System.getProperty("javax.net.ssl.keyStore");
        String keyStorePassword = System.getProperty("javax.net.ssl.keyStorePassword");
        KeyStore keystore = KeyStore.getInstance(KeyStore.getDefaultType());
        try(FileInputStream input = new FileInputStream(keyStoreFilename)) {
            keystore.load(input, keyStorePassword.toCharArray());
        }
        KeyManagerFactory keyManagerFactory = KeyManagerFactory.getInstance(
            KeyManagerFactory.getDefaultAlgorithm());
        keyManagerFactory.init(keystore, keyStorePassword.toCharArray());

        SslContext context = GrpcSslContexts.configure(
            SslContextBuilder.forServer(keyManagerFactory)
            ).build();
        GrpcUsersController stub = new GrpcUsersController();

        Server server = NettyServerBuilder.forPort(PORT)
            .addService(stub).sslContext(context).build();

        String serverURI = String.format(SERVER_BASE_URI, InetAddress.getLocalHost().getHostName(), PORT, GRPC_CTX);

        Log.info(String.format("Users gRPC Server ready @ %s\n", serverURI));
        server.start().awaitTermination();
    }
}
```

The server can now be started as we did before for a gRPC server.

Executing the Servers

When the servers execute, they must know what is the file that contains their keystore and the password for that keystore.

For clients, and for servers that also execute requests to other servers, we must also know the file containing the truststore file and its password.

This information is conveyed through arguments to the java machine:

```
java -Djavax.net.ssl.keyStore=<keystore-filename>
     -Djavax.net.ssl.keyStorePassword=<keystore-password>
     -Djavax.net.ssl.trustStore=<truststore-filename>
     -Djavax.net.ssl.trustStorePassword=<truststore-password>
<mainClass>
```

Executing the Servers

When the servers execute, they must know what is the file that contains their keystore and the password for that keystore.

For clients, and for servers that also execute requests to other servers, we must also know the file containing the truststore file and its password.

This information is conveyed through arguments to the java machine:

```
java -cp sd2526.jar -Djavax.net.ssl.keyStore=users-domain-ser
      -Djavax.net.ssl.keyStorePassword=password
      -Djavax.net.ssl.trustStore=truststore.ks
      -Djavax.net.ssl.trustStorePassword=changeit lab8.server.rest.RestUsersServer
```

This command is for the REST server in a docker named users.

Executing the Servers

When the servers execute, they must know what is the file that contains their keystore and the password for that keystore.

For clients, and for servers that also execute requests to other servers, we must also know the file containing the truststore file and its password.

This information is conveyed through arguments to the java machine:

```
java -cp sd2526.jar -Djavax.net.ssl.keyStore=users-domain-ser
      -Djavax.net.ssl.keyStorePassword=password
      -Djavax.net.ssl.trustStore=truststore.ks
      -Djavax.net.ssl.trustStorePassword=changeit lab8.server.grpc.GrpcUsersServer
```

This command is for the gRPC server in a docker named users.

EXECUTING THE SERVERS

The docker file that generates the image with your code must also be modified to copy the keystore and truststore files to the image:

```
FROM ubuntu

RUN apt-get update && apt-get install iproute2 -y
ENV JAVA_HOME=/opt/java/openjdk
COPY --from=eclipse-temurin:17 $JAVA_HOME $JAVA_HOME
ENV PATH=$PATH:$JAVA_HOME/bin

# working directory inside docker image
WORKDIR /home/sd

# copy hibernate config
COPY hibernate.cfg.xml hibernate.cfg.xml
# copy an example image for experiments
COPY example.png example.png

# copy the jar created by assembly to the docker image
COPY target/sd*.jar sd2526.jar

# copy keystore and truststore
COPY *.ks /home/sd/

# copy the jar created by assembly to the docker image
COPY target/sd*.jar sd2526.jar

CMD ["java", "-cp", "sd2526.jar", "-Djavax.net.ssl.keyStore=/home/sd/users-server.ks" ,\
    "-Djavax.net.ssl.keyStorePassword=password" ,\
    "-Djavax.net.ssl.trustStore=/home/sd/truststore.ks" ,\
```

EXECUTING THE SERVERS

The docker file that generates the image with your code must also be modified to copy the keystore and truststore files to the image:

```
FROM ubuntu

RUN apt-get update && apt-get install iproute2 -y
ENV JAVA_HOME=/opt/java/openjdk
COPY --from=eclipse-temurin:17 $JAVA_HOME $JAVA_HOME
ENV PATH=$PATH:$JAVA_HOME/bin

# working directory inside docker image
WORKDIR /home/sd

# copy hibernate config
COPY hibernate.cfg.xml hibernate.cfg.xml
# copy an example image for experiments
COPY example.png example.png

# copy the jar created by assembly to the docker image
COPY target/sd*.jar sd2526.jar

# copy keystore and truststore
COPY *.ks /home/sd/

# copy the jar created by assembly to the docker image
COPY target/sd*.jar sd2526.jar
```

```
CMD ["java", "-cp", "sd2526.jar", "-Djavax.net.ssl.keyStore=/home/sd/users-server.ks" ,\
    "-Djavax.net.ssl.keyStorePassword=password" ,\
    "-Djavax.net.ssl.trustStore=/home/sd/truststore.ks" ,\
```

(Notice that the example in this week executes the gRPC Server as default, also providing the jvm variables for the keystore and truststore)

GOALS

In the end of this lab you should be able to:

- Understand what is HTTPS and SSL/TLS
- Know how to generate a keystore with server cryptographic keys
- Know how to generate a truststore with root certificates and the certificate for your server
- Know how to develop a REST server using https in Java
- Know how to develop a gRPC server using TLS in Java
- **Know how to modify your REST clients to use https**
- **Know how to modify your gRPC clients to use TLS**

REST Client Code and HTTPS

RestClient Example

```
public class RestClient {
    private static final int MAX_RETRIES = 3;
    private static final long RETRY_SLEEP = 10000;

    private static final int READ_TIMEOUT = 10000;
    private static final int CONNECT_TIMEOUT = 10000;

    final Logger logger;

    final URI serverURI;
    final Client client;
    final ClientConfig config;

    protected WebTarget target;

    public RestClient(URI serverURI, Logger logger) {
        this.serverURI = serverURI;
        this.logger = logger;

        this.config = new ClientConfig();

        this.config.property(ClientProperties.READ_TIMEOUT, READ_TIMEOUT);
        this.config.property(ClientProperties.CONNECT_TIMEOUT, CONNECT_TIMEOUT);

        this.client = ClientBuilder.newClient(config);

        this.target = client.target(serverURI);
    }
```

REST Client Code and HTTPS

RestClient Example

This shows the constructor of the RestClient (omitting the methods that effectively do the operations).

```
public class RestClient {
    private static final int MAX_RETRIES = 3;
    private static final long RETRY_SLEEP = 10000;

    private static final int READ_TIMEOUT = 10000;
    private static final int CONNECT_TIMEOUT = 10000;

    final Logger logger;

    final URI serverURI;
    final Client client;
    final ClientConfig config;

    protected WebTarget target;

    public RestClient(URI serverURI, Logger logger) {
        this.serverURI = serverURI;
        this.logger = logger;

        this.config = new ClientConfig();

        this.config.property(ClientProperties.READ_TIMEOUT, READ_TIMEOUT);
        this.config.property(ClientProperties.CONNECT_TIMEOUT, CONNECT_TIMEOUT);

        this.client = ClientBuilder.newClient(config);

        this.target = client.target(serverURI);
    }
```

There are no changes here, it is sufficient that the URI that is used to initialize the client starts with https:// instead of http:// and that the server is identified by its hostname instead of IP address.

gRPC Client Code and TLS

GrpcClient Example

```
public class GrpcClient {
    final Logger logger;
    final private URI serverURI;
    final protected Channel channel;

    protected GrpcClient(URI serverURI, Logger logger) {
        this.serverURI = serverURI;
        this.logger = logger;

        String trustStoreFilename = System.getProperty("javax.net.ssl.trustStore");
        String trustStorePassword = System.getProperty("javax.net.ssl.trustStorePassword");

        try {
            KeyStore trustStore = KeyStore.getInstance(KeyStore.getDefaultType());
            try (FileInputStream input = new FileInputStream(trustStoreFilename)) {
                trustStore.load(input, trustStorePassword.toCharArray());
            }

            TrustManagerFactory trustManagerFactory = TrustManagerFactory.getInstance(
                TrustManagerFactory.getDefaultAlgorithm());
            trustManagerFactory.init(trustStore);

            SslContext context = GrpcSslContexts.configure(
                SslContextBuilder.forClient().trustManager(trustManagerFactory)
                ).build();
            this.channel = NettyChannelBuilder
                .forAddress(serverURI.getHost(), serverURI.getPort())
                .sslContext(context)
                .enableRetry()
                .build();

        } catch (Exception e) {
            throw new RuntimeException("Failed to initialize SSL context", e);
        }
    }
```

gRPC Client Code and TLS

GrpcClient Example

```
public class GrpcClient {
    final Logger logger;
    final private URI serverURI;
    final protected Channel channel;

    protected GrpcClient(URI serverURI, Logger logger) {
        this.serverURI = serverURI;
        this.logger = logger;

        String trustStoreFilename = System.getProperty("javax.net.ssl.trustStore");
        String trustStorePassword = System.getProperty("javax.net.ssl.trustStorePassword");

        try {
            KeyStore trustStore = KeyStore.getInstance(KeyStore.getDefaultType());
            try (FileInputStream input = new FileInputStream(trustStoreFilename)) {
                trustStore.load(input, trustStorePassword.toCharArray());
            }

            TrustManagerFactory trustManagerFactory = TrustManagerFactory.getInstance(
                TrustManagerFactory.getDefaultAlgorithm());
            trustManagerFactory.init(trustStore);

            SslContext context = GrpcSslContexts.configure(
                SslContextBuilder.forClient()
                ).build();
            this.channel = NettyChannelBuilder
                .forAddress(serverURI.getHost(
                .sslContext(context)
                .enableRetry()
                .build();

        } catch (Exception e) {
            throw new RuntimeException("Failed to initialize SSL context", e);
        }
    }
```

We will start by accessing the values of the default JVM properties (that has shown before should be passed as an argument when starting the process) regarding both the filename that contains the truststore and the password that protects that truststore.

gRPC Client Code and TLS

GrpcClient Example

```
public class GrpcClient {
    final Logger logger;
    final private URI serverURI;
    final protected Channel channel;

    protected GrpcClient(URI serverURI, Logger logger) {
        this.serverURI = serverURI;
        this.logger = logger;

        String trustStoreFilename = System.getProperty("javax.net.ssl.trustStore");
        String trustStorePassword = System.getProperty("javax.net.ssl.trustStorePassword");

        try {
            KeyStore trustStore = KeyStore.getInstance(KeyStore.getDefaultType());
            try (FileInputStream input = new FileInputStream(trustStoreFilename)) {
                trustStore.load(input, trustStorePassword.toCharArray());
            }

            TrustManagerFactory trustManagerFactory = TrustManagerFactory.getInstance(
                TrustManagerFactory.getDefaultAlgorithm());
            trustManagerFactory.init(trustStore);

            SslContext context = GrpcSslContexts.configure(
                SslContextBuilder.forClient().t
                ).build();
            this.channel = NettyChannelBuilder
                .forAddress(serverURI.getHost()
                .sslContext(context)
                .enableRetry()
                .build();

        } catch (Exception e) {
            throw new RuntimeException("Failed
        }
    }
```

Similar to the server side, we will have to create a (initially empty) truststore – notice that a truststore is just a keystore, the main difference is that it only stores certificates with public keys of entities that we trust – that we will load with the content of the file containing the truststore, also providing the password that protects that file.

gRPC Client Code and TLS

GrpcClient Example

```
public class GrpcClient {
    final Logger logger;
    final private URI serverURI;
    final protected Channel channel;

    protected GrpcClient(URI serverURI, Logger logger) {
        this.serverURI = serverURI;
        this.logger = logger;

        String trustStoreFilename = System.getProperty("javax.net.ssl.trustStore");
        String trustStorePassword = System.getProperty("javax.net.ssl.trustStorePassword");

        try {
            KeyStore trustStore = KeyStore.getInstance(KeyStore.getDefaultType());
            try (FileInputStream input = new FileInputStream(trustStoreFilename)) {
                trustStore.load(input, trustStorePassword.toCharArray());
            }

            TrustManagerFactory trustManagerFactory = TrustManagerFactory.getInstance(
                TrustManagerFactory.getDefaultAlgorithm());
            trustManagerFactory.init(trustStore);

            SslContext context = GrpcSslContexts.configure(
                SslContextBuilder.forClient().trustManager(trustManagerFactory)
                ).build();
            this.channel = NettyChannelBui
                .forAddress(serverURI.getH
                .sslContext(context)
                .enableRetry()
                .build();

        } catch (Exception e) {
            throw new RuntimeException("Fa
        }
    }
```

Instead of a KeyManagerFactory, for the client side we instead need a TrustManagerFactory, that we can instanciate using its factory and using default cryptographic algorithms.
We then have to initialize this trustManagerFactory with the information that we loaded to our truststore.

gRPC Client Code and TLS

GrpcClient Example

```
public class GrpcClient {
    final Logger logger;
    final private URI serverURI;
    final protected Channel channel;

    protected GrpcClient(URI serverURI, Logger logger) {
        this.serverURI = serverURI;
        this.logger = logger;

        String trustStoreFilename = System.getProperty("javax.net.ssl.trustStore");
        String trustStorePassword = System.getProperty("javax.net.ssl.trustStorePassword");

        try {
            KeyStore trustStore = KeyStore.getInstance(KeyStore.getDefaultType());
            try (FileInputStream input = new
                trustStore.load(input, trust
            }

            TrustManagerFactory trustManager
                TrustManagerFactory.getDefau
            trustManagerFactory.init(trustStore);

            SslContext context = GrpcSslContexts.configure(
                SslContextBuilder.forClient().trustManager(trustManagerFactory)
                ).build();
            this.channel = NettyChannelBuilder
                .forAddress(serverURI.getHost(), serverURI.getPort())
                .sslContext(context)
                .enableRetry()
                .build();

        } catch (Exception e) {
            throw new RuntimeException("Failed to initialize SSL context", e);
        }
    }
```

We can now generate a SslContext for clients providing to it the trustManagerFactory that we have just created and initialized.

gRPC Client Code and TLS

GrpcClient Example

```
public class GrpcClient {
    final Logger logger;
    final private URI serverURI;
    final protected Channel channel;

    protected GrpcClient(URI serverURI, Logger logger) {
        this.serverURI = serverURI;
        this.logger = logger;

        String trustStoreFilename = System.ge
        String trustStorePassword = System.ge

        try {
            KeyStore trustStore = KeyStore.ge
            try (FileInputStream input = new
                trustStore.load(input, trustS
            }

            TrustManagerFactory trustManagerF
                TrustManagerFactory.getDefaul
            trustManagerFactory.init(trustSto

            SslContext context = GrpcSslConte
                SslContextBuilder.forClient()
                ).build();
            this.channel = NettyChannelBuilder
                .forAddress(serverURI.getHost(), serverURI.getPort())
                .sslContext(context)
                .enableRetry()
                .build();
        } catch (Exception e) {
            throw new RuntimeException("Failed to initialize SSL context", e);
        }
    }
```

And now we can create the communication channel that we will be using the interact with the remote gRPC server. Notice that we are now using the NettyChannelBuilder (instead of the ManagedChannelBuilder as we were doing before).
The options are the same (including the enableRetry option) but now we must provide also the SSL Context that contains the certificates with the public keys of entities that we trust.

gRPC Client Code and TLS

GrpcUsersClient Example

```
public class GrpcUsersClient extends GrpcClient implements Users {
    private static Logger Log = Logger.getLogger(GrpcUsersClient.class.getName());

    final GrpcUsersGrpc.GrpcUsersBlockingStub stub;

    public GrpcUsersClient(URI serverURI) {
        super( serverURI, Log);
        stub = GrpcUsersGrpc.newBlockingStub( channel);
    }
```

Using the TLS enriched channel we can instantiate the client stub exactly as we did before.

Executing the Clients

When the clients execute, they must know what is the file that contains the truststore and the password for that truststore.

Similar to the servers this information is conveyed through arguments to the java machine (but only those that refer to the truststore):

(it is similar for REST and gRPC)

```
java -cp sd2526.jar -Djavax.net.ssl.trustStore=truststore.ks
      -Djavax.net.ssl.trustStorePassword=changeit lab8.clients.CreateUserClient
```

Executing the Clients

When the clients execute, they must know what is the file that contains the truststore and the password for that truststore.

Similar to the servers this information is conveyed through arguments to the java machine (but only those that refer to the truststore):

This is the example to execute the client that creates Users (it can interact with both REST and gRPC servers) in a docker container on the same network as the server.

```
java -cp sd2526.jar -Djavax.net.ssl.trustStore=truststore.ks
     -Djavax.net.ssl.trustStorePassword=changeit lab8.clients.CreateUserClient
```

Bonus: Regarding the Second Project

In the project your servers interact with each other, so you need each server to have a keystore and a truststore.

- You can use the same keystore for REST and gRPC servers.
- Servers must have both a keystore and a truststore (including all Root CA certificates and the certificate of the server itself)

Bonus: Regarding the Second Project

Does using HTTPS makes your server secure?

Bonus: Regarding the Second Project

Does using HTTPS makes your server secure?

Not exactly since clients can still execute the operations that are used by the server internally to do things as propagating messages among servers.

But using HTTPS allows us to enrich the security of those operations by adding a mandatory password that is shared among all servers to authenticate them. Since communication now is encrypted, these passwords will not be transmitted in cleartext.

EXERCISE (1/2)

In the code that supports this lab you will find an adapted version of the code from lab5 a keystore, truststore, (and the temporary certificate for the Users server). The cryptographic material was generated assuming the server will be executed in a containers named "users".

1. Use this code and cryptographic material to test both the REST and gRCP Users server with secure communication channels.
2. Launch the server container: `docker run -it --name users.domain -h users.domain --network sdnet sd2526-lab8-xxxxx-yyyyy /bin/bash`
3. In that container start one of the servers: e.g.,

```
java -cp sd2526.jar -Djavax.net.ssl.keyStore=users-domain-server.ks -Djavax.net.ssl.keyStorePassword=password lab8.server.grpc.GrpcUsersServer
```

4. Launch another container in the same network: `docker run -it --network sdnet sd2526-lab8-xxxxx-yyyyy /bin/bash`
5. In the second container execute the client with the correct URL: e.g.,

java -cp `sd2526.jar -Djavax.net.ssl.trustStore=truststore.ks -Djavax.net.ssl.trustStorePassword=changeit lab8.clients.CreateUserClient grpc://users.domain:9000/grpc nuno pwd Nuno domain example.png 233223432`

EXERCISE (2/2)

- After validating that you can run the provided example in both REST and GRPC:

1. Delete the files providing the truststore, keystore, and server certificate (truststore.ks, users-domain-server.ks, users-domain.cert)
2. Use the instructions in the start of these slides to generate new keystore and truststore.
3. Verify that you can use your generated files to run the provided example (by generating again the docker image and going over the previous steps).
4. After this you can start modifying all servers (and clients) in your project to support secure communication.